

Assignment -23

Name: K.Shalini

Batch: 37

Roll No: 2303A52311

Lab: 23.2

Task 1 – Project Proposal

- Students choose a real-world problem to solve with an AI-assisted application (e.g., e-commerce billing system, health monitoring dashboard, student attendance tracker).
- Prepare a project proposal including:
 - o Objective
 - o Features
 - o Tools/technologies to be used
 - o Expected output

Prompt:

#“Suggest 5 simple real-world software project ideas for a student using web technologies. Include problem and features.”

```
#Prepare a project proposal for student attendance tracker including:  
# Objective, Features, Tools/technologies to be used, Expected output
```

Output:

```
#“Suggest 5 simple real-world software project ideas for a student using  
web technologies. Include problem and features.”  
1. Personal Budget Tracker  
    - Problem: Many people struggle to keep track of their expenses  
and manage their budgets effectively.  
    - Features:  
        - User authentication and profile management
```

- Expense tracking with categories (e.g., food, transportation, entertainment)

- Monthly budget setting and alerts when nearing limits
- Data visualization with charts and graphs to show spending patterns

2. **Recipe Sharing Platform**

- **Problem**: Home cooks often look for new recipes and want to share their own

- **Features**:
 - User registration and profile creation
 - Recipe submission with ingredients, instructions, and photos
 - Search and filter recipes by cuisine, dietary restrictions, and preparation time
 - Commenting and rating system for user feedback

3. **Event Planning Application**

- **Problem**: Planning events can be stressful and disorganized without a proper tool.

- **Features**:
 - Event creation with details (date, time, location, description)
 - Guest list management and RSVP tracking
 - Task assignment and reminders for event preparation
 - Integration with calendar apps for scheduling

4. **Online Study Group Platform**

- **Problem**: Students often need a collaborative space to study and share resources.

- **Features**:
 - User registration and group creation
 - Discussion forums for different subjects or topics
 - Resource sharing (e.g., notes, links, files)
 - Real-time chat for group communication

5. **Local Business Directory**

- **Problem**: People often struggle to find local businesses and services in their area.

- **Features**:
 - Business listing with categories (e.g., restaurants, shops, services)
 - User reviews and ratings for businesses
 - Search and filter options based on location, category, and ratings
 - Map integration to show business locations

```
#Prepare a project proposal for student attendance tracker including:
# Objective, Features, Tools/technologies to be used, Expected output
**Project Proposal: Student Attendance Tracker**
- **Objective**: The objective of this project is to develop a web-based
application that allows teachers to easily track and manage student
attendance. The application will provide an efficient way for teachers to
mark attendance, generate reports, and communicate with students and
parents regarding attendance records.
- **Features**:
    1. User Authentication: Teachers can create accounts and log in
securely to access the attendance tracking system.
    2. Student Management: Teachers can add, edit, and delete student
profiles, including their names, contact information, and class details.
    3. Attendance Marking: Teachers can mark attendance for each student
on a daily basis, with options for present, absent, late, or excused.
    4. Attendance Reports: The application will generate reports that
summarize attendance records for individual students and the entire class
over specified time periods.
    5. Notifications: The system will send notifications to students and
parents regarding attendance status and any absences.
- **Tools/Technologies to be Used**:
    - Frontend: HTML, CSS, JavaScript (React.js or Vue.js for a more
dynamic interface)
    - Backend: Node.js with Express.js for server-side logic
    - Database: MongoDB or MySQL for storing student and attendance data
    - Authentication: JWT (JSON Web Tokens) for secure user authentication
    - Deployment: Heroku or Netlify for hosting the application
    - **Expected Output**: The expected output of this project is a fully
functional web application that allows teachers to manage student
attendance efficiently. The application will provide an intuitive
interface for marking attendance, generating reports, and sending
notifications. It will help improve communication between teachers,
students, and parents regarding attendance records and contribute to
better attendance management in educational institutions.
```

Task 2 – Application Design

- Use AI tools to assist in:
 - o Designing system architecture diagrams.
 - o Defining database schema (if needed).
 - o Creating wireframes/UI mockups for frontend design.

Prompt:

```
#"Design a system architecture for a [student attendance tracker]. Include frontend, backend, database and explain flow."  
#create a database schema (if needed).
```

```
#Create wireframes/UI mockups for frontend design.
```

Output:

```
#"Design a system architecture for a [student attendance tracker]. Include frontend, backend, database and explain flow."  
System Architecture for Student Attendance Tracker:  
1. Frontend:  
    - The frontend will be a web application built using React.js. It will provide an intuitive user interface for students and teachers to interact with the attendance tracker.  
    - The main features of the frontend will include:  
        - Student login and registration  
        - Teacher login and registration  
        - Dashboard for students to view their attendance records  
        - Dashboard for teachers to manage attendance and view reports  
        - Real-time notifications for students and teachers regarding attendance status  
2. Backend:  
    - The backend will be developed using Node.js with Express.js framework. It will handle all the business logic and API endpoints for the application.  
    - The main functionalities of the backend will include:  
        - User authentication and authorization for students and teachers  
        - CRUD operations for managing attendance records  
        - Generating reports and analytics for attendance data  
        - Sending real-time notifications to users  
3. Database:  
    - The database will be a relational database, such as PostgreSQL, to store user information, attendance records, and other relevant data.
```

- The database schema will include tables for users (students and teachers), attendance records, and notifications.

Flow:

1. User Registration and Login:

- Students and teachers will register on the frontend by providing their details. The backend will handle the registration process, store user information in the database, and return a success response.
- Upon successful registration, users can log in to the system. The backend will authenticate the user credentials and generate a JWT token for session management.

2. Attendance Management:

- Teachers can create attendance records for their classes. They can mark students as present, absent, or late. The backend will store these records in the database.
- Students can view their attendance records on their dashboard. The frontend will fetch this data from the backend and display it in a user-friendly format.

3. Real-time Notifications:

- The backend will use WebSockets to send real-time notifications to students and teachers regarding attendance.
- For example, if a student is marked absent, they will receive a notification on their dashboard. Similarly, teachers will receive notifications about attendance updates and reports.

4. Reporting and Analytics:

- Teachers can generate reports on attendance data, such as monthly attendance summaries or individual student attendance records. The backend will process the data and return the results to the frontend for display.
- The frontend will present these reports in a visually appealing manner, using charts and graphs to help teachers analyze attendance trends.

Overall, this system architecture provides a comprehensive solution for tracking student attendance, allowing for efficient management and real-time updates for both students and teachers.

#create a database schema (if needed).

Database Schema:

1. Users Table:

- id (primary key, auto-increment)
- username (varchar, unique)
- password (varchar)
- role (enum: 'student', 'teacher')

- created_at (timestamp)

- updated_at (timestamp)

2. Attendance Records Table:

- id (primary key, auto-increment)

- student_id (foreign key referencing Users.id)

- teacher_id (foreign key referencing Users.id)

- date (date)

- status (enum: 'present', 'absent', 'late')

- created_at (timestamp)

- updated_at (timestamp)

3. Notifications Table:

- id (primary key, auto-increment)

- user_id (foreign key referencing Users.id)

- message (varchar)

- is_read (boolean)

- created_at (timestamp)

- updated_at (timestamp)

This database schema allows for efficient storage and retrieval of user information, attendance records, and notifications. The relationships between the tables ensure data integrity and facilitate the necessary operations for the student attendance tracker system.

#Create wireframes/UI mockups for frontend design.

Wireframes/UI Mockups for Frontend Design:

1. Login Page:

- A simple login form with fields for username and password, along with a login button. There will also be a link to the registration page for new users.

2. Registration Page:

- A registration form with fields for username, password, and role selection (student or teacher). There will be a register button to submit the form.

3. Student Dashboard:

- A dashboard that displays the student's attendance records in a table format, showing the date and status (present, absent, late). There will also be a section for notifications.

4. Teacher Dashboard:

- A dashboard that allows teachers to manage attendance records. It will include a form to create new attendance records, a table to view existing records, and a section for generating reports. There will also be a notifications section for teachers.

5. Attendance Management Page:

- A page where teachers can mark attendance for their classes. It will include a list of students with checkboxes to mark them as present, absent, or late. There will be a submit button to save the attendance records.

6. Reports Page:

- A page where teachers can generate and view attendance reports. It will include options to filter reports by date range, class, or individual students. The reports will be displayed in a visually appealing format using charts and graphs.

These wireframes/UI mockups provide a clear visual representation of the frontend design for the student attendance tracker system. They ensure that the user interface is intuitive and user-friendly for both students and teachers, allowing them to easily navigate and interact with the application.

Task 3 – AI-Assisted Code Development

- Implement the application in phases:
 - o Frontend: Web/mobile UI with HTML/CSS/JS (or framework).
 - o Backend: RESTful APIs or server logic.
 - o Database: Schema design, SQL queries.
 - o Integration: API calls, authentication, error handling.

Prompt:Generate a full-stack application in phases:

1. Frontend: HTML, CSS, JavaScript UI for a simple app (e.g., To-Do or Student Manager).
2. Backend: Python (Flask) REST API for handling requests.
3. Database: SQLite schema and SQL queries.
4. Integration: Connect frontend with backend using API calls, include authentication and error handling.

Provide code for each part separately.

Code:

```
# Full-stack Student Manager: Flask backend with embedded frontend and SQLite database.
# Run this script with Flask installed: pip install flask

import sqlite3
```

```

from pathlib import Path
from flask import Flask, jsonify, request, render_template_string

DB_PATH = Path(__file__).with_suffix('.db')

app = Flask(__name__)

HTML_PAGE = """<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Manager</title>
    <style>
        body { font-family: Arial, sans-serif; background: #f4f7fc;
margin: 0; padding: 24px; }
        .container { max-width: 600px; margin: 0 auto; background: white;
padding: 24px; border-radius: 12px; box-shadow: 0 10px 30px
rgba(0,0,0,0.08); }
        h1 { margin-top: 0; }
        form input, form button { display: block; width: 100%; margin-
bottom: 12px; padding: 10px; border: 1px solid #ccc; border-radius: 6px; }
        form button { background: #007bff; color: white; border: none;
cursor: pointer; }
        form button:hover { background: #0056cc; }
        .student-card { padding: 12px; border-bottom: 1px solid #eee;
display: flex; justify-content: space-between; align-items: center; }
        .student-card:last-child { border-bottom: none; }
        .actions button { margin-left: 8px; }
        .actions button { padding: 6px 10px; border: none; border-radius:
6px; cursor: pointer; }
        .actions .delete { background: #dc3545; color: white; }
        .actions .update { background: #28a745; color: white; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Student Manager</h1>
        <form id="studentForm">
            <input type="text" id="name" placeholder="Name" required>
            <input type="number" id="age" placeholder="Age" required>
            <input type="text" id="grade" placeholder="Grade" required>
            <button type="submit">Add Student</button>

```



```

    </form>
    <div id="studentList"></div>
</div>

<script>
    async function fetchStudents() {
        const res = await fetch('/api/students');
        const students = await res.json();
        const list = document.getElementById('studentList');
        list.innerHTML = '';
        students.forEach(student => {
            const card = document.createElement('div');
            card.className = 'student-card';
            card.innerHTML = `
                <div><strong>${student.name}</strong> • Age
${student.age} • Grade ${student.grade}</div>
                <div class="actions">
                    <button class="update"
onclick="updateStudent(${student.id})">Update</button>
                    <button class="delete"
onclick="deleteStudent(${student.id})">Delete</button>
                </div>
            `;
            list.appendChild(card);
        });
    }

    async function addStudent(event) {
        event.preventDefault();
        const name = document.getElementById('name').value.trim();
        const age = Number(document.getElementById('age').value);
        const grade = document.getElementById('grade').value.trim();

        if (!name || !grade || age <= 0) {
            alert('Please enter valid student information.');
```

```

        document.getElementById('studentForm').reset();
        fetchStudents();
    }

    async function deleteStudent(id) {
        await fetch(`/api/students/${id}`, { method: 'DELETE' });
        fetchStudents();
    }

    async function updateStudent(id) {
        const name = prompt('Enter updated name:');
        if (!name) return;
        const age = Number(prompt('Enter updated age:'));
        const grade = prompt('Enter updated grade:');
        if (!name.trim() || age <= 0 || !grade?.trim()) {
            alert('Invalid update values.');
```

return;

```

        }
        await fetch(`/api/students/${id}`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ name: name.trim(), age, grade:
grade.trim() })
        });
        fetchStudents();
    }

    document.getElementById('studentForm').addEventListener('submit',
addStudent);
    fetchStudents();
</script>
</body>
</html>
"""

```

```

def get_db_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

```

```

def init_db():
    conn = get_db_connection()
    conn.execute(
        '''CREATE TABLE IF NOT EXISTS students (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            age INTEGER NOT NULL,
            grade TEXT NOT NULL
        )'''
    )
    conn.commit()
    conn.close()

@app.route('/')
def index():
    return render_template_string(HTML_PAGE)

@app.route('/api/students', methods=['GET'])
def list_students():
    conn = get_db_connection()
    rows = conn.execute('SELECT * FROM students').fetchall()
    conn.close()
    return jsonify([dict(row) for row in rows])

@app.route('/api/students', methods=['POST'])
def add_student():
    data = request.get_json() or {}
    name = data.get('name', '').strip()
    age = data.get('age')
    grade = data.get('grade', '').strip()

    if not name or not isinstance(age, int) or age <= 0 or not grade:
        return jsonify({'error': 'Invalid student data'}), 400

    conn = get_db_connection()
    cursor = conn.execute(
        'INSERT INTO students (name, age, grade) VALUES (?, ?, ?)',
        (name, age, grade)
    )
    conn.commit()

```

```

        student_id = cursor.lastrowid
        conn.close()
        return jsonify({'id': student_id, 'name': name, 'age': age, 'grade':
grade}), 201

@app.route('/api/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    data = request.get_json() or {}
    name = data.get('name', '').strip()
    age = data.get('age')
    grade = data.get('grade', '').strip()

    if not name or not isinstance(age, int) or age <= 0 or not grade:
        return jsonify({'error': 'Invalid student data'}), 400

    conn = get_db_connection()
    cursor = conn.execute(
        'UPDATE students SET name = ?, age = ?, grade = ? WHERE id = ?',
        (name, age, grade, student_id)
    )
    conn.commit()
    updated = cursor.rowcount
    conn.close()

    if updated == 0:
        return jsonify({'error': 'Student not found'}), 404

    return jsonify({'id': student_id, 'name': name, 'age': age, 'grade':
grade})

@app.route('/api/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    conn = get_db_connection()
    cursor = conn.execute('DELETE FROM students WHERE id = ?',
(student_id,))
    conn.commit()
    deleted = cursor.rowcount
    conn.close()

    if deleted == 0:
        return jsonify({'error': 'Student not found'}), 404

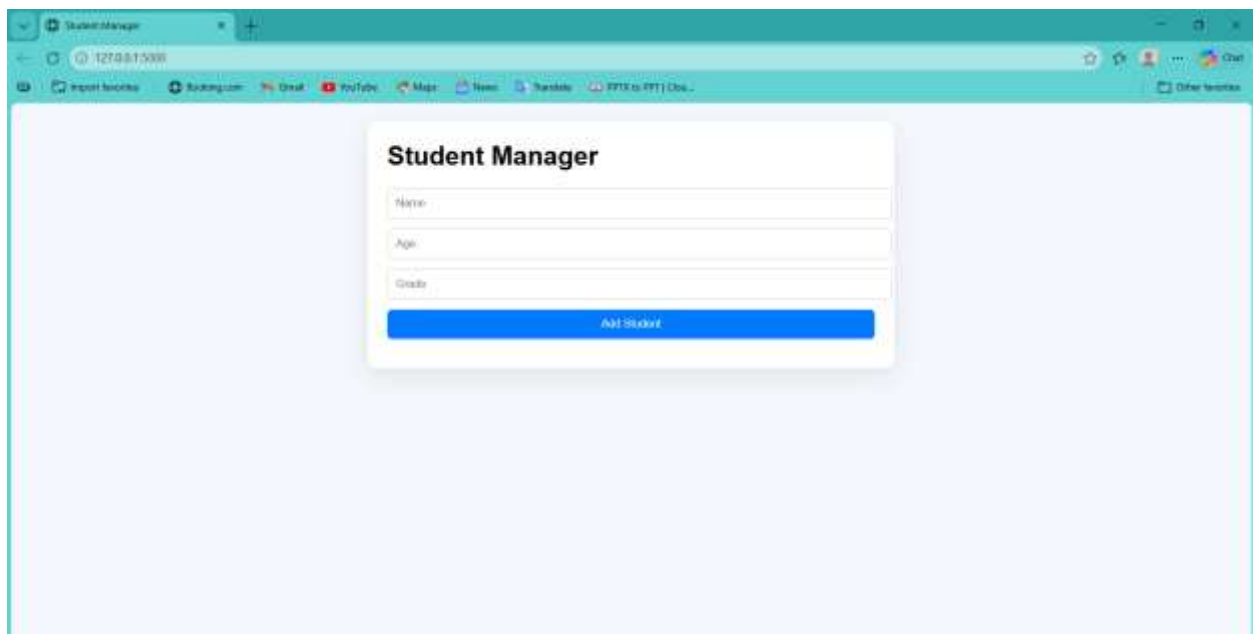
```

```
    return jsonify({'status': 'deleted'})

if __name__ == '__main__':
    init_db()
    app.run(debug=True, port=5000)
```

Output:

```
PS C:\Users\Vaishwarya Nema\puri> python lab23.py
* Serving Flask app 'lab23'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-647-757
127.0.0.1 - - [06/Apr/2026 16:14:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 16:14:17] "GET /api/students HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 16:14:17] "GET /Favicon.ico HTTP/1.1" 404 -
```



Task 4 – Documentation & Code Quality

- Generate automatic documentation and inline comments using AI.
- Ensure clear README.md for project setup and usage.
- Conduct AI-assisted code review to refine readability and Maintainability.

Prompt:Generate proper documentation for a Python project including inline comments, function descriptions, and a README.md file with installation steps, usage instructions, and features. Also review the code and suggest improvements for readability and maintainability.

Output:

```
# To-Do App

## Features
- Add tasks
- View tasks

## Setup
1. Install Flask:
    pip install flask
2. Run backend:
    python app.py
3. Open index.html in browser

## Usage
Enter task → Click Add → View list
```

Task 5 – Testing & Debugging

- Use AI to:
 - o Generate unit tests.
 - o Identify vulnerabilities/security flaws.
 - o Suggest optimizations for performance.

Prompt: Generate unit tests for a Python application using unittest or pytest. Identify possible bugs or vulnerabilities in the code and suggest fixes. Also recommend performance optimizations.

Code:

```
from flask import Flask, request, jsonify
import sqlite3
app = Flask(__name__)
# Database setup
conn = sqlite3.connect("tasks.db", check_same_thread=False)
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS tasks (task TEXT)")
```

```

# ☒ Home route (prevents 404 error)
@app.route("/")
def home():
    return """
    <h2>To-Do App is Running ☒ Add task
@app.route("/add", methods=["POST"])
def add_task():
    data = request.get_json()

    if not data or "task" not in data:
        return jsonify({"error": "Task required"}), 400

    task = data["task"]
    cursor.execute("INSERT INTO tasks VALUES (?)", (task,))
    conn.commit()

    return jsonify({"message": "Task added"}), 200

# ☒ Get tasks
@app.route("/tasks", methods=["GET"])
def get_tasks():
    cursor.execute("SELECT task FROM tasks")
    tasks = [row[0] for row in cursor.fetchall()]
    return jsonify(tasks), 200

# ☒ IMPORTANT: Prevent auto-run during testing
if __name__ == "__main__":
    app.run(debug=True)

```

Output:

```
PS C:\Users\Vaishwarya Nemaipuri> python lab23.py
* Serving Flask app 'lab23'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-647-757
127.0.0.1 - - [06/Apr/2026 16:17:45] "GET / HTTP/1.1" 200 -
* Detected change in 'c:\Users\Vaishwarya Nemaipuri\lab23.py', reloading
* Restarting with stat
PS C:\Users\Vaishwarya Nemaipuri>
```



Task 6 – Deployment & Presentation

- Deploy application on a cloud/free hosting platform (Heroku, GitHub Pages, Firebase, etc.).
- Prepare a final presentation including:
 - o Problem statement
 - o Solution architecture
 - o Live demo
 - o Reflection on AI assistance and limitations

Expected Outcomes:

- A complete, deployable software application.
- Students gain experience in end-to-end development with AI tools.
- Improved skills in teamwork, documentation, and ethical AI usage.

Prompt: Explain how to deploy a web application on platforms like GitHub Pages, Heroku, or Firebase. Provide steps for deployment and create a presentation outline including problem statement, architecture, demo, and AI usage reflection.

Code:

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

conn = sqlite3.connect("tasks.db", check_same_thread=False)
```



```

cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS tasks (task TEXT)")

@app.route("/")
def home():
    return "☑ To-Do App Deployed Successfully"

@app.route("/add", methods=["POST"])
def add_task():
    data = request.get_json()

    if not data or "task" not in data:
        return jsonify({"error": "Task required"}), 400

    cursor.execute("INSERT INTO tasks VALUES (?)", (data["task"],))
    conn.commit()
    return jsonify({"message": "Task added"}), 200

@app.route("/tasks", methods=["GET"])
def get_tasks():
    cursor.execute("SELECT task FROM tasks")
    tasks = [row[0] for row in cursor.fetchall()]
    return jsonify(tasks), 200

# For deployment platforms
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

Output:

```

PS C:\Users\Aishwarya.Remalipuri> python lab23.py
* Serving Flask app "lab23"
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.6:5000
Press CTRL+C to quit
127.0.0.1 - - [06/Apr/2026 16:19:14] "GET / HTTP/1.1" 200 -
192.168.1.6 - - [06/Apr/2026 16:19:24] "GET / HTTP/1.1" 200 -
192.168.1.6 - - [06/Apr/2026 16:19:24] "GET /favicon.ico HTTP/1.1" 404 -

```

